Privacy-Conscious Synthetic Test Environments — De-Identification & Re-ID Risk

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

What this solution is . . . 4

How it works . . . 5

The deployment model — Assess → Design → Generate/Build → Validate → Expand . . . 5

Benefits . . . 6

Limitations (read honestly) . . . 6

Requirements 6

S1. A documented field classification for the in-scope dataset . . . 7

S2. An agreed disclosure-risk threshold (the release gate) . . . 7

S3. A governed, read-only, minimized source extract (de-identification route only) . . . 7

S4. A pseudonymization / key-management decision . . . 8

S5. An evidence destination and retention decision . . . 8

Global Requirements — Synthetic Data & AI Test-Enablement Solutions 8

1. A Python data/engineering environment . . . 9

2. A target clinical data model / standard . . . 9

3. An isolated non-production test environment . . . 9

4. Privacy, governance & compliance stakeholders and policy . . . 9

5. AI/QA use cases and a pilot scope . . . 10

6. Reference/real-data access for fidelity (read-only, minimized) . . . 10

Implementation 10

Stage 0 — Preconditions . . . 11

Stage 1 — Assess . . . 11

Stage 2 — Design . . . 11

Stage 3 — Generate / Build . . . 12

Stage 4 — Validate (the release gates) . . . 12

Stage 5 — Expand . . . 12

Rollback . . . 13

Testing & Validation 13

T1. Field classification completeness . . . 13

T2. Direct-identifier suppression check . . . 13

T3. k-anonymity gate . . . 13

T4. l-diversity gate (sensitive-attribute protection) . . . 14

T5. Disclosure-risk / inference gate . . . 14

T6. Fidelity / coverage (utility survived de-identification) . . . 14

T7. Evidence-record completeness . . . 14
T8. Release-decision gate (human) . . . 15
Troubleshooting 15
The re-ID risk assessment FAILs on k-anonymity (smallest class < k) . . . 15
l-diversity FAILs even though k passes . . . 15
Disclosure-risk proportion is over threshold but k looks fine . . . 15
High `unique_record_share` on the synthesis route . . . 16
`phi_field_scanner.py` audit finds residual SSN/email/phone in the output . . . 16
`deid_pipeline.py` refuses to run ("key inside the test environment") . . . 16
Fidelity/coverage FAILs — distributions collapsed . . . 16
`reid_risk_assessor.py` errors: "classification lists no quasi-identifiers" . . . 16
The metrics all PASS — is the data releasable? . . . 16
Expansion dataset behaves differently from the pilot . . . 17
Best Practices 17
Keep the determination separate from the method . . . 17
Define "safe" as a number before you generate . . . 17
Classification is the highest-leverage decision . . . 17
Never let real PHI touch the shared environment . . . 17
Default to suppression; pseudonymize only with a documented need . . . 17
Treat the fidelity/privacy trade-off explicitly . . . 17
Watch for memorization on the synthesis route . . . 18
Make every released dataset traceable . . . 18
Re-run the gates on every expansion . . . 18
Roll back cleanly on failure . . . 18
Defer version-specific vendor behavior . . . 18
Deliverables Summary 18
Documentation . . . 18
The 5 Python tools . . . 19
What it adds up to . . . 19
Appendix A — PowerShell Scripts 20

Executive Summary

AI and QA teams need realistic clinical test data — but test environments built from clinical data must be **privacy-safe**, and the organization must be able to *prove* it on review. Assertions are not enough; governance needs evidence.

Outcome. This solution establishes **privacy-conscious test environments** where the data is **safe by design and defensible on review** — either de-identified from a governed source or fully synthetic, with **re-identification risk assessed under an agreed threshold** and the supporting **privacy evidence captured**. AI teams evaluate models and QA exercises systems **without carrying real-PHI exposure into a shared environment**.

Approach. A pilot-first method — **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** — with explicit pass/fail gates. It classifies every field (direct identifier / quasi-identifier / sensitive / non-identifying), applies a configurable de-identification or synthesis pipeline, and then measures re-identification risk with **k-anonymity, l-diversity, disclosure-risk, and membership/attribute- inference** signals against a threshold agreed with privacy. The result is packaged into a structured **evidence record** — classification → transformation → metric → threshold → result — the artifact governance signs against. Five Python tools automate the work: a PHI-field scanner, a de-identification pipeline, a fidelity/coverage report, a re-ID-risk assessor, and a config/evidence helper — all **safe by default** (no real PHI written to shared locations; dry-run where they write).

Why it is safe. Real source data stays read-only and minimized in the privacy-controlled zone; only *released* datasets enter the isolated test environment; any re-identification key is held outside it. Disclosure risk is **measured, not assumed**, and a candidate that fails a gate is rolled back, not released. Critically, the solution keeps a clean separation of duties: engineering produces the **method and evidence**, while the **regulatory determination** — HIPAA Safe Harbor / Expert Determination, IRB/privacy sign-off, residual-risk acceptance — is **deferred to the organization's privacy, compliance, and legal functions**. A technical PASS is a signal for that decision, not a substitute for it.

Bottom line. Test data that is safe by design and defensible on review — de-identified or synthetic, re-ID risk assessed under threshold, evidence captured — so AI teams can evaluate without privacy exposure, and governance can approve with confidence.

Solution Overview

What this solution is

Test environments built from clinical data must be **privacy-safe** — and the organization must be able to *demonstrate* that they are. This solution establishes **privacy-conscious test environments** for AI and QA teams by combining three things: a **de-identification (and/or synthesis) pipeline**, a **re-identification-risk assessment** against a documented threshold, and the **privacy evidence** that governance needs to review and approve the release of the data.

The result is test data that is **safe by design and defensible on review** — either de-identified from a governed source or fully synthetic, with disclosure risk assessed under an agreed threshold and the supporting evidence captured — so AI teams can evaluate models and QA can exercise systems **without carrying real-PHI exposure into a shared test environment**.

> ***Boundary — read this first.*** *This solution provides the* ***technical method and the evidence****. It does* ***not*** *make the* ***regulatory or legal determination****. Whether a dataset qualifies as de-identified under a given regime (e.g. HIPAA Safe Harbor or Expert Determination), whether the residual disclosure risk is *acceptable*, and whether an IRB/privacy-office sign-off is required — those are decisions for the organization's* ***privacy, compliance, and legal*** *functions. The tools here compute k-anonymity, l-diversity, disclosure-risk and inference signals and package them into an evidence record; a qualified determiner reads that evidence and makes the call. Nothing in this solution should be read as a de-identification certification or legal advice.*

How it works

The solution treats "privacy-safe test data" as two complementary routes, chosen per dataset with the privacy stakeholders:

1. **De-identification route** — start from a **governed, read-only, minimized** source extract and

transform it (suppress direct identifiers, generalize quasi-identifiers, and reduce disclosure risk) until the re-identification-risk assessment passes the agreed threshold. Real source data is handled only under the organization's privacy controls and is **never** written into the shared test environment in identifiable form.

1. **Synthesis route** — generate **fully artificial** records against a target clinical data model

(e.g. FHIR / OMOP CDM / the source schema) so no real individual's record is present at all. The same risk assessment is still run to check for memorization/leakage (e.g. membership or attribute inference) before release.

Both routes converge on the same **evidence record**: the classification of every field, the transformations applied, the risk metrics computed, the threshold they were measured against, and the pass/fail result — the artifact governance signs against.

Target model

Where the solution touches a schema, it works against whatever **clinical data model / standard** the AI/QA teams have selected as in scope — **FHIR**, **OMOP CDM**, or the source system's native schema. The scripts are model-agnostic (they operate on tabular exports / column classifications), so the same pipeline serves whichever target the organization documents in requirements.

The deployment model — Assess → Design → Generate/Build → Validate → Expand

Every solution in this category follows the same pilot-first model with explicit pass/fail gates and a documented rollback:

- **Assess** — profile a governed sample; classify each field (direct identifier / quasi-identifier /

sensitive attribute / non-identifying); agree the **disclosure-risk threshold** and route (de-identify vs. synthesize) with the privacy stakeholders.

- **Design** — define the **De-Identification Pipeline – Pilot – v1**: the per-field transformation

policy (suppression, generalization, date shifting, pseudonymization) and the risk metrics + target threshold that will gate release.

- **Generate / Build** — run the pipeline (or the synthesizer) against the pilot scope to produce the

candidate test dataset in the isolated environment.

• **Validate** — run the **Re-ID Risk Assessment** and the **fidelity/coverage** checks; compare to the pass/fail gates; capture the **Privacy Evidence Record**. If a gate fails, **roll back** (do not release) and tighten the policy.

• **Expand** — only after the pilot passes and privacy/governance signs the evidence, widen scope to further domains or service lines, re-running the same gates each time.

Benefits

• **Defensible on review** — governance receives a structured evidence record (classification → transformation → risk metric → threshold → result), not an assertion.

• **Safe by design** — the isolated environment never receives identifiable real PHI; disclosure risk is measured, not assumed.

• **Repeatable** — the pipeline and thresholds are captured as config, so the same discipline applies every time scope expands.

• **Separation of duties** — engineering produces method + evidence; the privacy/legal function owns the determination. The two are cleanly decoupled.

Limitations (read honestly)

• **Not a legal determination.** Passing the threshold is a *technical* signal; the regulatory call belongs to privacy/compliance/legal.

• **Risk is never zero.** k-anonymity, l-diversity and disclosure-risk metrics reduce and *quantify* risk; they do not eliminate it. Residual-risk acceptance is a governance decision.

• **Metrics depend on correct classification.** If a quasi-identifier is misclassified as non-identifying, the risk assessment understates exposure — classification review is part of the method.

• **Fidelity vs. privacy is a trade-off.** Stronger generalization lowers re-ID risk but reduces analytic utility; the pilot exists to find an acceptable point on that curve.

• **Vendor specifics are deferred.** Where a commercial synthesizer or de-identification platform is used, verify version-specific behavior against the vendor's documentation; this solution integrates such tools at the pattern level.

Requirements

This solution builds on the six **category-wide** prerequisites in `global-requirements-synthetic.md` (a Python data environment, a target clinical data model, an isolated non-production environment, privacy/governance stakeholders and policy, AI/QA use cases and a pilot scope, and read-only/minimized reference access for fidelity). **Verify those six first** — they are not repeated here.

Below are the prerequisites **specific to privacy-conscious de-identification and re-ID-risk work**. Confirm each before running the implementation steps.

S1. A documented field classification for the in-scope dataset

• **Description:** A per-field classification of the in-scope dataset into **direct identifiers** (name, MRN, SSN, contact details), **quasi-identifiers** (dates, ZIP/geography, age, sex, admission dates — the fields that combine to re-identify), **sensitive attributes** (diagnoses, procedures, results the disclosure of which is the harm), and **non-identifying** fields. This becomes the artifact **Field Classification – Pilot – v1**.

• **Why required:** Every downstream metric depends on it — quasi-identifiers drive k-anonymity, sensitive attributes drive l-diversity, direct identifiers drive suppression. A misclassified field silently invalidates the risk assessment.

• **How to verify:** Confirm every column in the pilot extract has exactly one classification and that the privacy stakeholder has reviewed the quasi-identifier list (the highest-leverage call).

• **How to implement if missing:** Run the profiling/classification tool (`phi_field_scanner.py`) to produce a **draft** classification, then have the privacy stakeholder review and correct it. The scanner *suggests*; the human decides.

S2. An agreed disclosure-risk threshold (the release gate)

• **Description:** The **quantitative threshold** the dataset must meet to be releasable into the test environment — e.g. a minimum **k** (equivalence-class size), a minimum **l** (distinct sensitive values per class), and/or a maximum acceptable **disclosure-risk** proportion — agreed with privacy and recorded as **Risk Threshold – Pilot – v1**.

• **Why required:** "Safe" must be defined as a number *before* generating, or the validation gate is subjective. The threshold is a governance decision the method measures against.

• **How to verify:** Confirm the threshold values and their owner are written down and approved.

• **How to implement if missing:** Facilitate the threshold decision with privacy/compliance (the solution provides the method to measure against it; it does **not** set the number).

S3. A governed, read-only, minimized source extract (de-identification route only)

• **Description:** Where the route is **de-identification** (not pure synthesis), a **minimized, read-only **extract of the real source — only the in-scope fields, only the pilot cohort — obtained and handled under the organization's privacy controls, staged** outside** the shared test environment.

• **Why required:** You cannot de-identify data you cannot read; but real data must never enter the shared harness in identifiable form. The extract is the *input* to the pipeline, not a test artifact.

• **How to verify:** Confirm the extract is minimized, read-only, access-logged, and staged in the privacy-controlled zone — not the AI/QA test environment.

• **How to implement if missing:** Request the minimized extract through the governed data-access process; if it cannot be obtained, use the **synthesis route** instead.

S4. A pseudonymization / key-management decision

• **Description:** A decision on how direct identifiers are handled — **suppressed/dropped** vs. **pseudonymized** with a mapping key — and, if a key exists, **where it is held, who controls it, and that it never enters the test environment**. Recorded as part of **De-Identification Pipeline – Pilot – v1**.

• **Why required:** A re-identification key stored alongside "de-identified" data defeats the entire exercise; this must be an explicit, governed decision, not a default.

• **How to verify:** Confirm the handling choice per direct identifier and, if pseudonymized, that the key custody is documented and segregated from the test environment.

• **How to implement if missing:** Default to **suppression** unless a governed re-linkage need is documented; if pseudonymized, place the key under the privacy function's custody.

S5. An evidence destination and retention decision

• **Description:** Where the **Privacy Evidence Record** is stored, who can read it, and how long it is retained — the audit trail governance will consult on review and on any future challenge.

• **Why required:** The evidence is the deliverable governance signs against; it must be durable, attributable, and findable — not a transient file in the test environment.

• **How to verify:** Confirm the evidence store, its access controls, and the retention period.

• **How to implement if missing:** Agree a governed evidence location (the reporting tool writes the record; the organization owns where it lands).

Global Requirements — Synthetic Data & AI Test-Enablement Solutions

These requirements apply to **every** solution in the **Synthetic** category (synthetic clinical data, hospital-twin simulation, and safe AI test enablement). Verify each before starting a solution's steps.

> ***Safety, vendor & regulatory boundary (applies to every Synthetic solution).*** *These solutions generate and validate **synthetic** (artificial) data and run in **isolated non-production** test environments. **Never place real Protected Health Information (PHI) into a shared synthetic harness, twin, or AI test environment** — where real data is touched at all it is read-only, minimized, and handled under the governing privacy controls. **Synthetic-data and digital-twin tools** (e.g. Synthea, and commercial platforms) and **AI/model frameworks** are integrated at the pattern/architectural level; specific vendor console internals are **cited or deferred** to that vendor's*

documentation, never invented. ***Regulatory and legal interpretation*** *(HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, disclosure risk acceptance, and any ITAR/NERC/NRC or contract constraints) is* ***deferred to the organization's privacy, compliance, and legal functions*** *— the solutions provide the technical method and evidence, not legal determinations.*

1. A Python data/engineering environment

• **Description:** A supported **Python** environment (e.g. 3.10+) with the standard data toolchain (data frames, statistics, and the synthetic-data / validation libraries the solution uses).

• **Why required:** The automation and reusable assets are authored in Python (generation, de-identification, fidelity/utility metrics, scenario runners).

• **How to verify:** Confirm Python + the required packages install and run in the target environment.

• **How to implement if missing:** Provision the environment and install dependencies before starting.

2. A target clinical data model / standard

• **Description:** A defined **clinical data model or interoperability standard** to generate against — e.g. **FHIR**, **OMOP CDM**, or the source system's own schema — plus the fields/domains in scope (patients, encounters, conditions, medications, labs, vitals, notes, etc.).

• **Why required:** Realistic synthetic data and a credible hospital twin require a target schema and clinically plausible relationships; validation is measured against it.

• **How to verify:** Confirm the target model/standard and the in-scope domains with the AI/QA teams.

• **How to implement if missing:** Select and document the target model before generating.

3. An isolated non-production test environment

• **Description:** A **segregated, non-production** environment (compute + storage) for the synthetic harness, hospital twin, and AI test runs — separate from production and from real-PHI systems.

• **Why required:** Safe evaluation of AI solutions before broader adoption depends on isolation; it also bounds any residual disclosure risk.

• **How to verify:** Confirm the environment is isolated, access-controlled, and not connected to production PHI stores.

• **How to implement if missing:** Stand up / obtain the isolated environment before running.

4. Privacy, governance & compliance stakeholders and policy

• **Description:** The governing **privacy, data-governance, and compliance** policies and the named stakeholders (privacy officer, security, and — where real source data or de-identification claims are involved — IRB/legal) whose requirements the work must satisfy.

• **Why required:** Regulated healthcare data constraints, de-identification acceptance, and enterprise data governance are policy decisions the technical method must align to, not defaults.

• **How to verify:** Confirm the applicable policies, disclosure-risk threshold, and approvers.

• **How to implement if missing:** Identify the policies and stakeholders before generating or releasing test data.

5. AI/QA use cases and a pilot scope

• **Description:** The **AI use cases and QA scenarios** the synthetic data / twin must support, and a **pilot scope** to validate on before broader rollout.

• **Why required:** Fidelity, coverage, and scenario design are driven by the AI use cases; every solution follows an **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model proven on a pilot.

• **How to verify:** Confirm the target AI use cases, evaluation goals, and a named pilot.

• **How to implement if missing:** Elicit the use cases and define the pilot before building the harness.

6. Reference/real-data access for fidelity (read-only, minimized)

• **Description:** Where synthetic **fidelity/utility** must be measured against reality, **read-only, minimized** access to reference statistics or a governed real dataset (under the privacy controls in §4) — or agreed reference distributions where real data cannot be used.

• **Why required:** Fidelity and validation assets compare synthetic output to real-world distributions and relationships; without a reference the claims are unfounded.

• **How to verify:** Confirm the reference source and its access/governance terms.

• **How to implement if missing:** Agree the reference approach (governed real stats vs. published reference distributions) before validating.

*The deployment model across every Synthetic solution is the same: **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, with explicit pass/fail (fidelity/privacy) gates and a documented rollback. Automation and reusable assets are delivered as **Python** — never real PHI is written to a shared harness, twin, or AI test environment.*

Implementation

Follow these ordered steps to stand up a privacy-conscious test environment on a **pilot** scope, with explicit pass/fail gates and a documented rollback. The steps follow the house model **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**. Every artifact is **named** with the convention `<Purpose> – <Scope> – <Stage>`; reuse those exact names in testing, troubleshooting, and scripts.

> ***Boundary reminder.*** *These steps produce **method and evidence**. The **determination** that the result is "de-identified" / the residual risk is acceptable belongs to privacy/compliance/legal. Do not release on a green metric alone — release on a green metric **plus** the recorded sign-off.*

Stage 0 — Preconditions

1. Confirm the six global requirements and the solution-specific prerequisites **S1–S5** in `02-requirements.md` are met. Do not start until **Risk Threshold – Pilot – v1** (S2) exists — the gate must be defined before you generate.

1. Confirm the **isolated non-production environment** is ready and is **not** connected to production PHI stores. This environment receives only synthetic or de-identified output — never the raw extract.

1. Confirm the Python environment is available (`python --version` ≥ 3.10). Install `pandas`/`numpy` if the scripts will use them (`pip install pandas numpy`).

Stage 1 — Assess

1. Obtain the governed input: either the **minimized, read-only source extract** (S3, de-ID route, staged in the privacy-controlled zone) **or** confirm the **synthesis route** and its target model.

1. Produce a **draft classification**: run `python phi_field_scanner.py --input <extract.csv> --out field-classification-draft.json`. This *suggests* direct-identifier / quasi-identifier / sensitive / non-identifying per column.

1. Have the **privacy stakeholder review and correct** the draft, especially the quasi-identifier list. Save the reviewed result as the artifact **Field Classification – Pilot – v1** (`field-classification-pilot-v1.json`).

1. Confirm **Risk Threshold – Pilot – v1** (S2): the minimum **k**, minimum **l**, and/or maximum disclosure-risk the dataset must meet to be releasable.

> ***Gate A (scope gate):*** *every in-scope column is classified and reviewed, and the threshold is recorded. If not, stop here.*

Stage 2 — Design

1. Build the pipeline configuration with the config helper:

`python deid_config_helper.py --classification field-classification-pilot-v1.json --k 5 --l 2 --out deid-pipeline-pilot-v1.json`. This generates a starting **De-Identification Pipeline – Pilot – v1** config: per-field policy — **suppress** direct identifiers, **generalize** quasi-identifiers (age→age band, ZIP→3-digit, dates→shifted/coarsened), pass through non-identifying fields.

1. Apply the **S4 pseudonymization/key decision** into the config: default direct identifiers to `suppress`; only set `pseudonymize` where a governed re-linkage need is documented and the key is held outside the test environment.

1. Review the generalization policy for utility: confirm the AI/QA use cases can still be exercised after generalization (this is the fidelity-vs-privacy trade-off). Record the reviewed config.

> ***Gate B (design gate):*** *the pipeline config exists, the direct-identifier handling matches the S4 decision, and no re-identification key is destined for the test environment.*

Stage 3 — Generate / Build

1. Produce the candidate dataset in the isolated environment:

• **De-ID route:** `python deid_pipeline.py --input <extract.csv> --config deid-pipeline-pilot-v1.json --out deidentified-pilot-v1.csv` (writes only under `--apply`; dry-run by default).

• **Synthesis route:** generate fully artificial records against the target model (via the selected synthesizer at the pattern level — see the vendor's documentation for platform specifics), landing `synthetic-pilot-v1.csv`.

1. Name the output **De-Identified Dataset – Pilot – v1** (or **Synthetic Dataset – Pilot – v1**).

This candidate is **not yet releasable** — it must pass Stage 4 first.

Stage 4 — Validate (the release gates)

1. Run the re-identification-risk assessment against the threshold:

`python reid_risk_assessor.py --input deidentified-pilot-v1.csv --classification field-classification-pilot-v1.json --k 5 --l 2 --out reid-risk-pilot-v1.json`. It reports **k-anonymity** (smallest equivalence class), **l-diversity**, the **disclosure-risk** proportion (share of records in classes smaller than k), and simple **membership/attribute- inference** signals, each with a **PASS/FAIL** against **Risk Threshold – Pilot – v1**.

1. Run the fidelity/coverage check (utility must survive de-identification):

`python fidelity_coverage_report.py --synthetic deidentified-pilot-v1.csv --reference <ref-stats.json> --out fidelity-pilot-v1.json`.

1. Assemble the **Privacy Evidence Record – Pilot – v1**:

`python deid_config_helper.py --assemble-evidence --classification field-classification-pilot-v1.json --pipeline deid-pipeline-pilot-v1.json --risk reid-risk-pilot-v1.json --fidelity fidelity-pilot-v1.json --out privacy-evidence-pilot-v1.md`. Store it at the S5 evidence destination.

> ***Gate C (privacy release gate):*** *the re-ID assessment is **PASS** against every threshold and the evidence record is captured. If any metric **FAILs**, **roll back** — do **not** release the dataset. Return to Stage 2, tighten the generalization/suppression policy, and re-run Stages 3–4.*

1. **Route the evidence to privacy/compliance/legal for the determination and sign-off.** Only after the recorded sign-off is the dataset releasable into the shared test environment. (Technical PASS ≠ regulatory determination.)

Stage 5 — Expand

1. With the pilot passed and signed, widen scope one increment at a time (a further domain, service

line, or larger cohort). For each increment, **re-run Stages 1–4** and capture a new evidence record — expansion never inherits the pilot's PASS.

1. Keep the pipeline config, thresholds, and evidence records under version control so every released

test dataset is traceable to the classification, transformations, and metrics that justified it.

Rollback

At any gate failure: **do not release** the candidate dataset; retain the failing evidence record (it documents *why* it was held); tighten the pipeline policy and repeat. Because the isolated environment only ever receives *released* datasets, a held candidate has zero exposure.

Testing & Validation

These validation procedures gate the release of a candidate dataset into the isolated test environment. Each has an explicit **expected result** and a **PASS/FAIL** meaning. The candidate is **not releasable** until the privacy gates pass **and** privacy/compliance/legal records a sign-off (technical PASS ≠ regulatory determination).

Run these against the named artifacts from `03-implementation.md`: **De-Identified Dataset – Pilot – v1** (or **Synthetic Dataset – Pilot – v1**), **Field Classification – Pilot – v1**, and **Risk Threshold – Pilot – v1**.

T1. Field classification completeness

- **Procedure:** Confirm every in-scope column in the candidate has exactly one classification in

Field Classification – Pilot – v1, and that direct identifiers listed there are **absent or transformed** in the output.

- **Expected / PASS:** No unclassified columns; no direct-identifier column survives in identifiable

form.

- **FAIL:** Any unclassified column, or any direct identifier present in cleartext — stop, fix the

classification/pipeline, do not proceed.

T2. Direct-identifier suppression check

- **Procedure:** Scan the candidate output with `phi_field_scanner.py` (`--mode audit`) to confirm no

values matching direct-identifier patterns (SSN, MRN, email, phone) remain.

- **Expected / PASS:** Zero direct-identifier matches in the released columns.
- **FAIL:** Any match → a suppression rule was missed; roll back to Stage 2.

T3. k-anonymity gate

- **Procedure:** `python reid_risk_assessor.py --input <candidate.csv> --classification field-classification-pilot-v1.json --k <threshold-k>`.

It computes the smallest equivalence class over the quasi-identifiers.

- **Expected / PASS:** Smallest equivalence-class size ≥ **k** from **Risk Threshold – Pilot – v1**

(i.e. every record is indistinguishable from at least k–1 others on the quasi-identifiers).

- **FAIL:** Any class smaller than k → generalize/suppress further; do not release.

T4. l-diversity gate (sensitive-attribute protection)

- **Procedure:** Same tool, `--l <threshold-l>`; checks each equivalence class contains at least **l** distinct sensitive-attribute values.

- **Expected / PASS:** Every class has ≥ **l** distinct sensitive values (attribute disclosure is bounded).

- **FAIL:** Any class below l → the class leaks the sensitive attribute; tighten and re-run.

T5. Disclosure-risk / inference gate

- **Procedure:** Same tool reports the **disclosure-risk proportion** (share of records in classes below k) and simple **membership/attribute-inference** signals (e.g. unique-record share, exact quasi-identifier matches to the reference).

- **Expected / PASS:** Disclosure-risk proportion ≤ the maximum in **Risk Threshold – Pilot – v1**; inference signals within tolerance.

- **FAIL:** Over threshold → roll back. (For the **synthesis route**, a high membership-inference signal indicates the synthesizer memorized real records — regenerate.)

T6. Fidelity / coverage (utility survived de-identification)

- **Procedure:** `python fidelity_coverage_report.py --synthetic <candidate.csv> --reference <ref-stats.json>`.

Compares marginal distributions and category coverage of the candidate to the agreed reference.

- **Expected / PASS:** Distributions and coverage within the documented fidelity tolerance — the AI/QA use cases can still be exercised.

- **FAIL:** Utility collapsed (over-generalized) → find a better point on the privacy/utility curve; this is a design decision, not necessarily a privacy failure.

T7. Evidence-record completeness

- **Procedure:** Confirm `privacy_evidence_report.py` produced **Privacy Evidence Record – Pilot – v1**

containing: the field classification, the transformations applied, the k/l/disclosure-risk metrics, the thresholds they were measured against, PASS/FAIL per gate, and the fidelity result.

- **Expected / PASS:** All sections present and stored at the S5 evidence destination.
- **FAIL:** Any missing section → regenerate; governance cannot review an incomplete record.

T8. Release-decision gate (human)

- **Procedure:** Route the evidence record to privacy/compliance/legal; confirm a **recorded sign-off** exists before releasing to the shared environment.
- **Expected / PASS:** Signed determination on file → dataset may be released.
- **FAIL / absent:** No sign-off → **not releasable**, regardless of green metrics. The determination is theirs, not the pipeline's.

Troubleshooting

Common failure modes when standing up a privacy-conscious test environment, and how to resolve them. Symptoms reference the named artifacts and tools from `03-implementation.md` and `05-testing.md`.

The re-ID risk assessment FAILs on k-anonymity (smallest class < k)

- **Cause:** Quasi-identifiers are too granular — a rare combination (e.g. exact date + 5-digit ZIP + age) isolates individuals.
- **Resolve:** In **De-Identification Pipeline – Pilot – v1**, strengthen generalization on the offending quasi-identifiers (wider age bands, ZIP truncation to 3 digits, coarser dates), re-run `deid_pipeline.py`, then `reid_risk_assessor.py`. Do **not** release until k ≥ threshold.

l-diversity FAILs even though k passes

- **Cause:** An equivalence class is large enough (k) but everyone in it shares the same sensitive value — the attribute leaks despite anonymity.
- **Resolve:** Coarsen the quasi-identifiers so classes merge and gain sensitive-value diversity, or reconsider whether that attribute should be released at this granularity. Re-run the assessor.

Disclosure-risk proportion is over threshold but k looks fine

- **Cause:** Many *small* classes just above the FAIL line still push the share of at-risk records over `max_disclosure_risk`. k is the minimum; disclosure-risk is a distribution.
- **Resolve:** Generalize the long tail of small classes; re-check both `k_anonymity` and `disclosure_risk` in the report.

High `unique_record_share` on the synthesis route

• **Cause:** The synthesizer memorized real records — a membership-inference signal, not just low utility.

• **Resolve:** Regenerate with the synthesizer's privacy/regularization controls (see the vendor's documentation for version-specific options). Treat this as a **privacy** failure, not a fidelity one.

`phi_field_scanner.py` audit finds residual SSN/email/phone in the output

• **Cause:** A direct-identifier column was misclassified as non-identifying, so the pipeline kept it.

• **Resolve:** Correct **Field Classification – Pilot – v1**, regenerate the pipeline config with `deid_config_helper.py`, re-run `deid_pipeline.py`, and re-audit. This is a **Gate A** classification miss — fix upstream, not by patching the output.

`deid_pipeline.py` refuses to run ("key inside the test environment")

• **Cause:** The config sets a field to `pseudonymize` with `key_location` inside the test environment — an S4 violation the pipeline blocks by design.

• **Resolve:** Move the re-identification key to the privacy-controlled zone (outside the test environment) and record its custody, or change the field to `suppress`. Then re-run.

Fidelity/coverage FAILs — distributions collapsed

• **Cause:** Over-generalization to pass the privacy gate destroyed analytic utility (the AI/QA use cases can no longer be exercised).

• **Resolve:** This is a **trade-off decision**, not automatically a privacy failure. Find a milder generalization that still passes `reid_risk_assessor.py`, or revise the reference tolerance with the stakeholders. Record the chosen point on the privacy/utility curve.

`reid_risk_assessor.py` errors: "classification lists no quasi-identifiers"

• **Cause:** The classification has no fields marked `quasi_identifier`, so k-anonymity is undefined.

• **Resolve:** Re-check **Field Classification – Pilot – v1** — dates, geography, and demographics are almost always quasi-identifiers. Have the privacy stakeholder confirm the list.

The metrics all PASS — is the data releasable?

• **Cause:** A green report is a **technical** signal only.

• **Resolve:** Route the **Privacy Evidence Record** to privacy/compliance/legal and obtain the recorded determination/sign-off (Testing **T8**). Release only after that. A technical PASS is never the regulatory determination.

Expansion dataset behaves differently from the pilot

- **Cause:** New scope has different cardinality/rarity, so the pilot's PASS does not carry over.
- **Resolve:** Re-run Stages 1–4 for the new increment and capture a fresh evidence record — expansion never inherits the pilot's result.

Best Practices

Durable practices for running privacy-conscious test environments. These keep the work **safe by design and defensible on review** as scope grows.

Keep the determination separate from the method

- Engineering produces **metrics + evidence**; privacy/compliance/legal makes the **determination**.

Never let a green k-anonymity number stand in for a regulatory sign-off. Wire the sign-off into the release gate so the two cannot be conflated.

Define "safe" as a number before you generate

- Agree **Risk Threshold – Pilot – v1** (minimum k, minimum l, max disclosure-risk) *before* building the pipeline. A threshold set after seeing results is not a control.

Classification is the highest-leverage decision

- Most re-ID risk hides in **misclassified quasi-identifiers** (dates, geography, rare categories).

Treat the classification as a reviewed artifact, not a script output — the scanner suggests, the privacy stakeholder decides.

Never let real PHI touch the shared environment

- The minimized source extract stays in the **privacy-controlled zone**; only *released* de-identified or synthetic datasets enter the isolated test environment. If a re-identification key exists, it is held **outside** that environment under the privacy function's custody.

Default to suppression; pseudonymize only with a documented need

- Suppress direct identifiers by default. Only pseudonymize where a governed re-linkage requirement is written down, and then segregate the key. A key stored beside the data defeats the exercise.

Treat the fidelity/privacy trade-off explicitly

- Stronger generalization lowers re-ID risk and lowers utility. Use the pilot to find an acceptable point on that curve, and record the choice. Don't over-generalize into uselessness or under- generalize into exposure by accident.

Watch for memorization on the synthesis route

• A synthesizer that memorizes real records fails privacy even though "no real data was copied."

Always run membership/attribute-inference checks on synthetic output, not just on de-identified output.

Make every released dataset traceable

• Version the classification, pipeline config, thresholds, and evidence record together. Any released test dataset should be reproducible from — and justified by — its captured evidence.

Re-run the gates on every expansion

• Expansion never inherits the pilot's PASS. Each new domain/cohort gets its own classification, risk assessment, and evidence record.

Roll back cleanly on failure

• On any gate failure, **do not release** — retain the failing evidence (it documents the hold), tighten the policy, and repeat. Because only released datasets enter the environment, a held candidate carries zero exposure.

Defer version-specific vendor behavior

• Where a commercial synthesizer or de-identification platform is used, integrate it at the pattern level and **verify version-specific behavior against the vendor's documentation** rather than assuming console internals.

Deliverables Summary

What the buyer receives — a complete, self-guided kit to establish a **privacy-conscious test environment**: de-identification/synthesis pipeline, re-identification-risk assessment against a threshold, and the privacy evidence governance needs to review and approve.

Documentation

• **Solution Overview** — the privacy model for test data (synthetic vs. de-identified), disclosure-risk concepts (k-anonymity, l-diversity, membership/attribute inference), the safe-by-design posture, and the Assess→Design→Generate/Build→Validate→Expand model.

• **Requirements Guide** — the six category-wide prerequisites plus five solution-specific ones (field classification, disclosure-risk threshold, governed source extract, pseudonymization/key decision, evidence destination), each with how to verify and how to implement.

• **Implementation Guide** — exact ordered steps, pilot-first, with named artifacts and the three pass/fail gates (scope, design, privacy release) plus rollback.

- **Testing & Validation** — eight validation procedures (classification, suppression, k-anonymity,

l-diversity, disclosure-risk/inference, fidelity, evidence completeness, human release decision), each with expected PASS/FAIL.

- **Troubleshooting** — common failure modes and resolutions.
- **Best Practices** — durable practices for keeping test data safe and defensible as scope grows.

The 5 Python tools

1. `phi_field_scanner.py` — *Assessment / profiling.* Profiles a tabular extract and **suggests** a

per-field classification (direct identifier / quasi-identifier / sensitive / non-identifying) using name and value-pattern heuristics; also runs an **audit** mode to detect residual direct-identifier values in an output. Produces the draft for **Field Classification – Pilot – v1**. Read-only; writes only a classification JSON.

1. `deid_pipeline.py` — *Transformation / generation.* Applies the de-identification pipeline —

suppress direct identifiers, generalize quasi-identifiers (age bands, ZIP truncation, date shifting/coarsening), pass through non-identifying fields — per the pipeline config. **Dry-run by default**; writes the de-identified dataset only with `--apply`.

1. `fidelity_coverage_report.py` — *Fidelity / coverage report.* Compares the candidate dataset's

marginal distributions and category coverage to an agreed reference-statistics file and reports whether utility survived de-identification, within a documented tolerance.

1. `reid_risk_assessor.py` — *Privacy / validation check.* Computes **k-anonymity**,

l-diversity, the **disclosure-risk proportion**, and simple **membership/attribute-inference** signals over the quasi-identifiers/sensitive attributes, and reports **PASS/FAIL** against the agreed threshold. The core release-gate tool.

1. `deid_config_helper.py` — *Scope / config helper.* Generates and validates a starting

De-Identification Pipeline – Pilot – v1 config from a field classification and target k/l, and (with `--assemble-evidence`) composes the **Privacy Evidence Record** from the classification, pipeline, risk, and fidelity outputs — so the pipeline, assessor, and evidence all read a consistent, versioned configuration.

What it adds up to

Test data that is **safe by design and defensible on review** — de-identified or synthetic, re-ID risk assessed under an agreed threshold, evidence captured — with the **regulatory determination cleanly deferred** to the organization's privacy, compliance, and legal functions.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

deid_config_helper.py

deid_pipeline.py

fidelity_coverage_report.py

phi_field_scanner.py

reid_risk_assessor.py